

Piloting XP on Four Mission-Critical Projects

Jerry Drobka, David Noftz, and Rekha Raghu, *Motorola*

According to proponents, Extreme Programming creates focused, efficient software development that heavily emphasizes customer involvement. As software developers for mission-critical two-way radio systems in Motorola's Commercial, Government, and Industrial Solutions Sector, we'd been hearing a lot about XP's benefits. We'd also heard stories about Web applications and small development projects that successfully used XP. The nagging questions remained, however. Would

XP work for us? Could developers in an established company use XP to develop complex mission-critical systems that could be maintained for decades? What would be the ramifications for quality and maintainability? For the purpose of this article, we'll assume that you're relatively familiar with the standard XP process. For detailed information regarding XP's roles, guidelines, and practices, read *Extreme Programming Explained*¹ and *Extreme Programming Installed*.²

Why Extreme Programming?

Typically, our development projects are part of large system releases with customer-enforced ship dates, set well before our customers finalize their product requirements. We also make staffing estimates—and product development teams start work—before the requirements are final. So, teams face the prospect of taking several months to develop an architecture that, because of changing requirements, is guaranteed to be wrong in some fashion. We hoped that XP would help us solve our changing requirements dilemma. We also hoped to improve productivity and decrease cycle time while maintaining our quality level.

Our process

We piloted XP with four development teams over an 18-month period, from product re-

Software development teams constantly battle to increase productivity while maintaining or improving quality. Four Motorola teams piloted Extreme Programming to see if it would let them satisfy their customers' constantly changing requirements while improving productivity.

quirements through product testing. In all cases, they followed object-oriented (OO) design principles and implemented the systems using C++.

Teams A, B, and C developed Windows-based embedded systems that acted as gateways. Team A's system was a gateway between a circuit-switched communications system and a packet-switched Internet-Protocol-based communication system. Team B's was a gateway between legacy telephone systems and a new IP-based communication system. Team C's was between legacy radio-frequency system interfaces and a new IP-based communication system. Team D's product, part of a Unix-based controller, was an application that provided arbitration and resource allocation.

Incorporating and adapting XP

XP is an agile, iterative development process quite different from the waterfall processes we were accustomed to using. We used XP's four key participant roles—customer, coach, manager, and developer—plus these development principles and practices:

- The planning game
- Short development cycles (in our case, each iteration was one calendar month long)
- Pair programming³
- Test-first programming
- Collective code ownership
- Frequent integration
- Never solving a problem that hasn't occurred
- Refactoring⁴
- Minimal documentation

XP's greatest asset is simplicity. Some purists might argue otherwise, but we don't believe that XP is a one-size-fits-all process.⁵ A few aspects didn't work with the type of product we were developing. So, we tailored the process slightly to better fit our project and organization.

The coach

When we started our pilot, we had no experience with XP. To ensure that we used this new process properly, we hired an experienced outside consultant as our coach. The coach helped us implement the process and tailor it for our organization and kept us from retreating to our "comfort zone" when the going got tough. Our coach also joined a team and pair programmed with our engineers, which was an effective on-the-job training method. We rec-

ommend that all teams experimenting with an agile process bring in an experienced coach. A consultant who's not a product of your organization's culture will bring a fresh perspective to your team and ease the transition.

The customer

XP requires a paying customer to fill the customer role, but our organization's structure made this impossible. Instead, each team used its technical domain expert as a customer proxy, who was responsible for gaining a complete understanding of the system's requirements. To accomplish this, these "customers" used personal knowledge and experience, worked with system-level engineers, interfaced with engineers from other product teams, and gained input from marketing groups. Behind the scenes, system-level engineers and marketing groups worked together to determine the overall system requirements. The customers took these inputs, wrote requirements when necessary, and provided the teams with use cases; they also worked with their managers to prioritize the features required for release. The development teams then planned their iterations on the basis of these use cases.

The planning game

At the beginning of each iteration, each team held a planning-game meeting. At this meeting, the manager supplied a list of functionalities (in the form of use cases) that he or she wanted to see completed during the next iteration. The developers, coach, and customer met for a four- to six-hour session to discuss this list. During this meeting, the team broke down each piece of functionality into small implementation tasks and assigned each task an effort estimate. The developers then totaled the estimates to determine if they could implement the requested functionality within a single iteration. If so, they assigned an owner to each implementation task. If not, they returned the list to the manager, who made the iteration's scope more realistic. The managers had the authority to move functionality from iteration to iteration. If they wished to change the project's scope, however, they had to obtain approval using Motorola's change control processes.

Architecture and documentation

When it comes to documentation, most developers fall into one of two camps: either you

Some purists might argue otherwise, but we don't believe that XP is a one-size-fits-all process.

We had to demonstrate to our senior management that XP offered tangible benefits over our existing processes.

fervently believe XP's creators when they say source code is the only necessary documentation, or you break into a cold sweat at the thought of no design artifacts. We're centrists on this issue, falling somewhere between these two extremes. We believe that source code gives the most accurate picture of system operation but that it doesn't tell you everything. For example, if you need to perform maintenance on an OO software system, how do you know where in the code to begin your investigation? Clearly, you need some sort of roadmap to give the developer a starting point, and then the source code takes over as the primary documentation. We used a high-level architecture document to provide this roadmap; it contained class diagrams, scenarios, and a process view of the system.⁵

Defining baseline architectures. Before we staffed the project, several pilot teams defined initial architectures to give the engineers a framework to use during the development's first iteration. A small group of technical experts created these baseline architectures in a four-hour blitz session. They designed and documented scenarios with the most customer impact and high architectural significance. These architectures also introduced key abstractions in the code and helped identify and define the system boundaries. We found that when the programming pairs had no architectural guidance during the project's first iteration, they created their own visions of what the overall architecture should be. These visions often differed considerably, resulting in disorganization and significant refactoring work.

Standardizing documentation. The software components our XP teams developed were part of a larger system that other teams were creating in sites across the country. Because each system component's requirements and scenarios had to be accessible to all the teams, we needed a common documentation format that everyone could understand. To facilitate communication with the other teams, we wrote use cases⁶ rather than user stories. This provided a common language and minimized our pilot's impact on other engineers and teams.

On all teams, the customers were responsible for writing and validating the use cases. They created a list of all possible use cases (prioritized by customer impact and architectural significance), elaborated the highest-priority

use cases, and delivered them to the development team. The customers wrote use cases in parallel with the development and delivered them in a just-in-time manner to the implementers. Because the customers wrote the use cases independently from the XP teams, we required that the use cases go through Motorola's formal technical-review process. This ensured that they were consistent with the overall system requirements and functionally correct for their respective software components.

Formal reviews

The products we develop have about a 30-year life span, during which engineers continuously upgrade and improve them. These engineers are almost never the ones who worked on the product originally, so maintainability's a big issue for us. We were concerned that XP's minimal documentation would make such work difficult. To mitigate this issue, we used design reviews to ensure maintainability.

At the end of each iteration, we updated our high-level architecture document to reflect the system's current state. We then scheduled a design review to evaluate the updated document. During the review, independent OO experts evaluated the design and pointed out inefficiencies or mistakes. People outside the team also verified the product's maintainability by evaluating the design and code using only the supplied documentation. These reviews helped us avoid overdocumenting our design.

XP requires the customer to write acceptance tests for the system at the story level, which the developers run periodically to verify the design's correctness. This wasn't feasible for us during implementation. Running the tests would have required a complete two-way radio system, and we only had our single system component. So, to ensure that the system met the customer's requirements, we held requirements verification reviews in which the customer was the key reviewer. The meetings also ensured that the existing unit and application-level test cases were sufficient to verify our software component's operation.

Our results

Even though our four teams followed slightly different processes because of their individual situations, they showed relatively consistent results. The teams collected data on qualitative and quantitative aspects of the process. We measured

the qualitative results with a survey that we distributed to the pilot's 29 participants.

Morale

XP is unique because it provides constant feedback to the developers. At the end of each day, our teams had a working product, which created a surge in morale. This constant feedback also helped the engineers gain a sense of accomplishment from their daily work; they could immediately see the positive impact their efforts had on the project. This is significant because when morale is high, people are excited about their work, leading to a more effective, efficient development team. Overall, 85 percent of the developers enjoyed using XP, 68 percent stated that using XP increased their job enjoyment, and 79 percent stated that, given a choice, they would definitely use XP again.

Learning

XP also reduced the learning curve for new team members. Having new team members sit in on pair-programming sessions for a few weeks was helpful. Once they actually began working in pairs, they learned the architecture quickly with help from the more experienced engineers with whom they were paired. This method also proved successful with new hires, who became productive after approximately one month of XP experience. In total, 55 percent of the developers believed that using XP shortened their initial project-learning curve.

Productivity

Our teams also experienced a significant increase in engineer productivity. We measured productivity using the formula (total KAELOC) / (total staff effort). KAELOC represents thousands of assembler-equivalent lines of code. Our total KAELOC value didn't include unit test code, and the total staff effort value included our refactoring effort. Using a productivity formula that includes lines of code can cause some problems, especially when extensive refactoring takes place.

Business needs drove our decision to use this formula. To justify the risk and expense of piloting a new software development process, we had to demonstrate to our senior management that XP offered tangible benefits over our existing processes. We satisfied this requirement by establishing a common denominator that would let us compare our results with previous projects' results. This productiv-

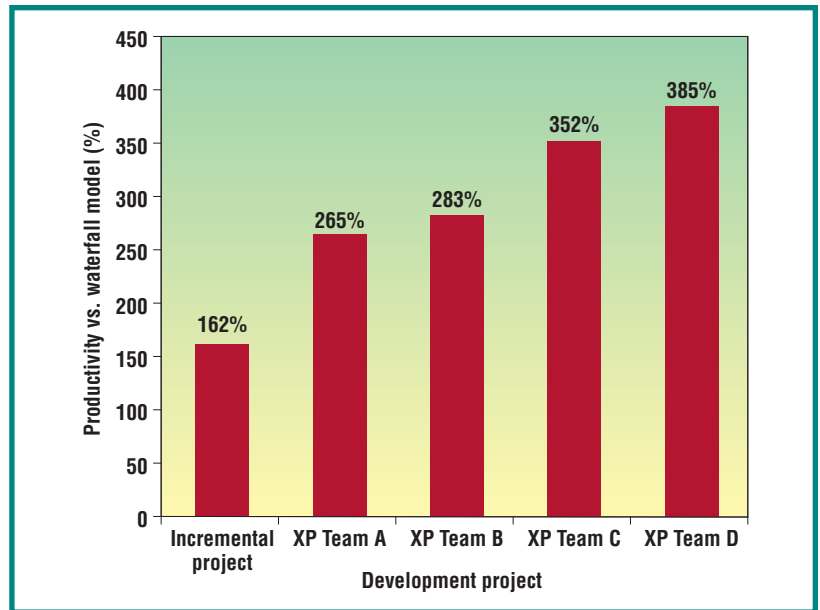


Figure 1. Productivity measurements.

ity calculation was available for all our historical projects. Figure 1 shows the percent improvement in our XP projects' productivity, compared to similar Motorola teams using different processes. Our teams showed high productivity levels according to this metric.

Test coverage

We also achieved excellent test coverage as an ancillary benefit of using XP. Figure 2 shows our teams' statement test coverage data. All four

Figure 2. Test coverage measurements.

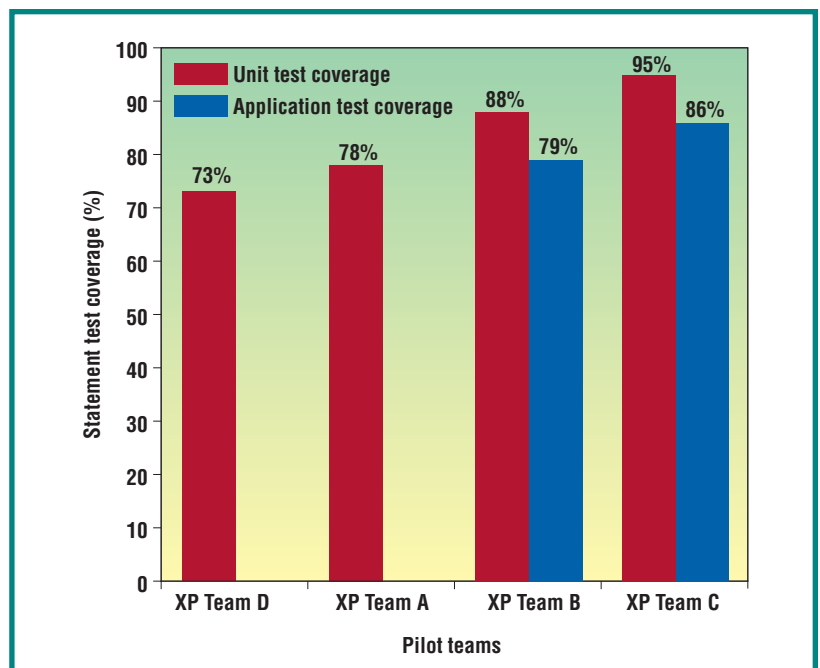
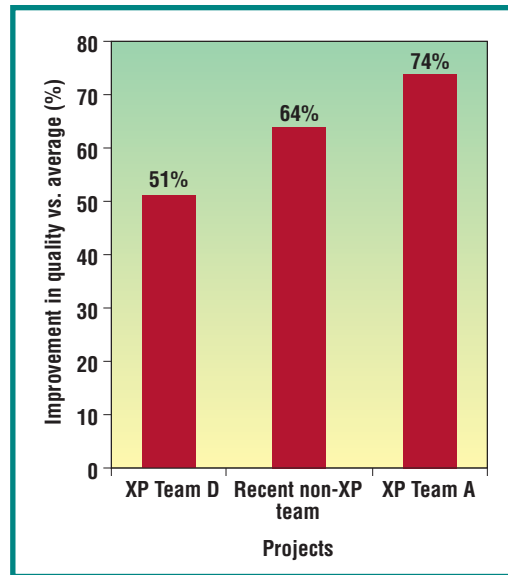


Figure 3. Quality measurements.



teams had unit test coverage data available, but only B and C measured application-level test coverage. Our teams' unit test coverage varied mainly because of how strictly they adhered to the test-first programming concept. Because Team C used test-first programming consistently, they obtained 95 percent unit test coverage and 85 percent application-level test coverage.

Quality and maintainability

Figure 3 shows Team A's and Team D's final quality results. These two projects have shipped; the others haven't gone through system testing, so no data is available. This chart shows the percent improvement in quality versus our division's average. We measured quality results using defect density, defined as (total number of defects found during system testing) / (total KAELOC). For our teams, KAELOC didn't include unit test code. Both teams' defect densities were significantly better than the division average. The XP teams in Figure 3 show quality levels similar to the non-XP team's, suggesting that XP projects can have quality levels comparable to those of projects created using other processes. As we mentioned before, our tailored XP process also had custom formal technical reviews built in to ensure the teams produced a maintainable design.

Eighty-two percent of the developers believed that the design and code they generated using XP were understandable, maintainable, and extensible. Eighty-eight percent believed

that the deliverables they produced were adequate for future development and maintenance. Finally, 80 percent were more confident in the design and code they generated while pair programming than when they work alone.

Sticking to the process

To get a feel for how well the teams were adhering to the process, we performed audits and held discussion sessions with the developers monthly. We found that the developers followed our modified XP process as specified approximately 80 percent of the time. Occasionally, both pair programmers weren't available at the same time. When this happened, the developer wrote unit test cases and then wrote the code when the other developer was available. One team didn't create a baseline architecture, which forced them to hold several extensive refactoring sessions. Another team didn't allocate a dedicated customer, so they rotated the team's senior members through the role. Another didn't consistently follow the test-first strategy, which resulted in us catching several defects during the acceptance test.

Bottom line: the teams tried their best to adhere to the process, but sometimes it wasn't possible. When this occurred, they tried to adapt the process to suit their needs, which sometimes worked and sometimes didn't.

Challenges

Although we view our XP pilot as a positive experience, the process presented some challenges. For example, pair programming requires large blocks of time. Experienced developers have busy schedules and often spend much of their day rushing from one meeting to another. They have a lot of trouble finding three to four hours of uninterrupted time each day to participate in pair programming. We found that the best way to alleviate this problem was to use "brute force"—we used a scheduling tool to block out time in their schedules.

Pair programming presents another problem. Managers must choose the pairs on their team carefully. Pair programming could be ineffective with shy or overbearing personalities, or with two relatively inexperienced developers. If a manager pairs a shy person with someone who's a bit overbearing, the shy person will have limited participation in the pair's

tasks. Also, pairing up inexperienced developers isn't desirable because they might not understand the development practices and how their task fits into the big picture.

We also noticed on several occasions that XP teams had difficulty interfacing with teams using traditional development processes. For instance, the XP team wanted just one or two pieces of the interface to work with at a time, but the traditional team wanted to develop and review the entire interface before providing it to the XP team. We found that defining common skeleton interfaces up front was an effective way to mitigate this issue.

XP team managers also face several challenges. Because so many tasks are performed collectively, managers have difficulty identifying high-performing and low-performing employees and must therefore pay closer attention than usual to developers' activities. In addition, because XP is so agile, managers might have difficulty determining how schedule changes will affect the project's anticipated end date.

Our pilot program's success has convinced us that you can use an agile process such as XP to develop complex mission-critical systems with long life cycles. An XP project's structure lets the team make rapid progress, even when facing volatile requirements. The productivity gains that XP provides also make it an attractive development process for most OO projects.

We feel obligated to warn first-time XP users, however, that integrating an agile process into a company with a culture favoring more traditional development processes isn't easy. You'll need to tailor the process to your circumstances. Some managers and developers will reject XP immediately because it's so different from the development processes they're accustomed to. All prospective XP users must undergo thorough training to obtain buy-in. Strong coaching is also necessary to ensure that the team adheres to the process and tailors it properly. Finally, periodic audits and code spot-checks by the system architect and coach help ensure that the team is using the process correctly. If you're willing to work hard and carefully introduce an agile process into your organization, your efforts should yield positive results. ☺

About the Authors



Jerry Drobka is a Distinguished Member of the Technical Staff at Motorola, where he's working on a next-generation two-way radio system. His research interests include requirements management in a project's initial stages. He earned his MS in electrical engineering from the Illinois Institute of Technology. Contact him at Motorola, 1301 E. Algonquin Rd., IL02/1923, Schaumburg, IL 60196; jerry.drobka@motorola.com.

David Noftz is an engineering group leader at Motorola, where he's a project manager developing a next-generation two-way radio system. His research interests include increasing productivity and reducing costs through software process improvement. He holds an MBA from DePaul University and an MS in computer science from the University of Illinois at Chicago. Contact him at Motorola, 1301 E. Algonquin Rd., IL02/1923, Schaumburg, IL 60196; david.noftz@motorola.com.



Rekha Raghu is a technical lead at Motorola, where she's working on the performance and capacity of a next-generation two-way radio dispatch system. Her research interests include agile processes and software process improvement. She received her MS in computer science from the University of Illinois at Chicago. She is a signatory and member of the Agile Alliance (<http://AgileAlliance.org>). Contact her at Motorola, 1301 E. Algonquin Rd., IL02/1923, Schaumburg, IL 60196; rekha.raghu@motorola.com.

References

1. K. Beck, *Extreme Programming Explained: Embrace Change*, Addison-Wesley, 1999.
2. R. Jeffries, A. Andersen, and C. Hendrickson, *Extreme Programming Installed*, Addison-Wesley, 2000.
3. L. Williams and R. Kessler, *Pair Programming Illuminated*, Addison-Wesley, 2002.
4. M. Fowler et al., *Refactoring: Improving the Design of Existing Code*, Addison-Wesley, 1999.
5. B. Boehm and R. Turner, *Balancing Agility and Discipline: A Guide for the Perplexed*, Addison-Wesley, 2003.
6. A. Cockburn, ed., *Writing Effective Use Cases*, Addison-Wesley, 2000.

For more information on this or any other computing topic, please visit our Digital Library at www.computer.org/publications/dlib.

Visit
Software
on the
Web



Log on to our Web site,
where you can

- Search our vast archives
- Preview upcoming topics
- Browse our calls for papers
- Submit your article for publication
- Subscribe or renew

www.computer.org/software